

COMPANION
to

ChatGPT Teaches TikZ

Chapter 1: Getting Started (B)

Contents

1.1	What This Companion Adds	2
1.2	Quick Review	3
1.3	Coordinates as Instructions	4
1.4	Building Longer Paths	6
1.5	Direction and Options	7
1.6	Repeating with <code>\foreach</code>	8
1.7	Debugging Workshop	9
1.8	Worked Project: A Coordinate Practice Board . . .	11
1.9	Exercises	13
1.10	Solutions	14
1.11	ChatGPT Workshop	16
1.12	Final Checklist	16

1.1 What This Companion Adds

Chapter 1 introduced the smallest useful TikZ vocabulary: the `tikzpicture` environment, `\draw`, coordinates, straight segments, options, arrows, semicolons, and `\foreach`. This Companion develops fluency with those ideas. It does not assume any material from later chapters.

The emphasis is on three habits:

- reading a TikZ command from left to right;
- changing one coordinate or option at a time; and
- recognizing syntax errors before changing the geometry.

The Companion is optional. If the examples in Chapter 1 are already comfortable, you may proceed directly to Chapter 2 and return here whenever you would like more practice.

Design

This Companion deliberately avoids nodes, labels, colors, shapes, and positioning. Those subjects belong to later chapters. Restricting the vocabulary makes the structure of every command easy to see.

1.2 Quick Review

Every picture has the same outer frame:

```
\begin{tikzpicture}
  drawing commands
\end{tikzpicture}
```

The basic drawing command is

```
\draw[options] (start) -- (end);
```

The option list may be omitted. The semicolon may not.

Item	Purpose
<code>tikzpicture</code>	Contains one complete TikZ picture.
<code>\draw</code>	Constructs and draws a path.
<code>(x,y)</code>	Specifies a coordinate.
<code>--</code>	Joins consecutive coordinates by a straight segment.
<code>[options]</code>	Changes the behavior or appearance of a command.
<code>-></code>	Places an arrowhead at the end of a path.
<code>\foreach</code>	Repeats a command for a list of values.

Item	Purpose
;	Ends a TikZ command.

Read `\draw[->] (0,0) -- (3,1);` as a sentence: draw, using an end arrow, from (0,0) by a straight segment to (3,1), and then end the command.

1.3 Coordinates as Instructions

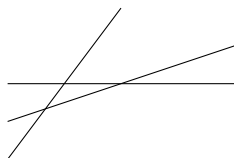
A coordinate does two jobs. It identifies a point, and it tells TikZ how far to move horizontally and vertically. The first number controls the horizontal position; the second controls the vertical position.

1.3.1 Changing One Coordinate

The following three segments begin at the origin. Only the endpoint changes.

```
\begin{tikzpicture}
  \draw (0,0) -- (3,0);
  \draw (0,-0.5) -- (3,0.5);
  \draw (0,-1) -- (1.5,1);
\end{tikzpicture}
```

See



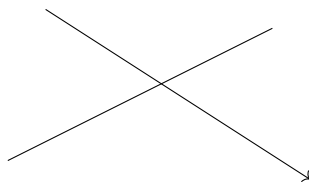
The first segment is horizontal. The second rises gently. The third rises more steeply because its horizontal change is smaller.

1.3.2 Negative and Decimal Coordinates

Coordinates need not be positive integers.

```
\begin{tikzpicture}
  \draw (-2,-0.5) -- (1.5,1.25);
  \draw[>-] (-1.5,1.5) -- (2,-0.75);
\end{tikzpicture}
```

See



Negative first coordinates lie to the left of the origin. Negative second coordinates lie below it. Decimal coordinates allow finer adjustments.

Design

When a line is almost correct, edit only one number. If several coordinates are changed at once, it becomes difficult to know which change improved or damaged the picture.

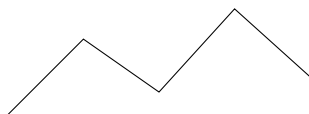
1.4 Building Longer Paths

A path may pass through several coordinates. Each occurrence of `--` adds one straight segment.

1.4.1 An Open Path

```
\begin{tikzpicture}  
  \draw  
    (0,0)  
    -- (1,1)  
    -- (2,0.3)  
    -- (3,1.4)  
    -- (4,0.5);  
\end{tikzpicture}
```

See



The line breaks in the source do not break the path. TikZ continues until it reaches the semicolon.

1.4.2 One Path or Several Commands?

Use one command when the segments form one continuous path:

```
\draw (0,0) -- (1,1) -- (2,0);
```

Use separate commands when the segments are independent:

```
\draw (0,0) -- (1,1);  
\draw (2,0) -- (3,1);
```

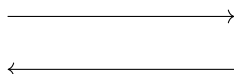
This distinction becomes important later, when options are applied to complete paths.

1.5 Direction and Options

The option `->` places an arrowhead at the final coordinate. Reversing the coordinate order therefore reverses the arrow.

```
\begin{tikzpicture}  
  \draw[->] (0,0) -- (3,0);  
  \draw[->] (3,-0.7) -- (0,-0.7);  
\end{tikzpicture}
```

See



The option list belongs immediately after `\draw`:

```
\draw[->] (0,0) -- (3,0);
```

Placing it at the end is incorrect:

```
\draw (0,0) -- (3,0) [->];
```

Common Mistake

The arrowhead belongs to the path, not to the final coordinate. Put the option before the first coordinate.

1.6 Repeating with `\foreach`

A loop replaces several nearly identical commands. In

```
\foreach \x in {0,1,2,3,4}
  \draw (\x,0) -- (\x,0.3);
```

TikZ substitutes each value in the list for `\x`. The drawing command is therefore performed five times.

1.6.1 Changing the Spacing

The list controls both the number and the positions of the repeated marks.

```
\begin{tikzpicture}
  \foreach \x in {0,0.5,1,1.5,2,2.5,3}
    \draw (\x,0) -- (\x,0.3);
\end{tikzpicture}
```

See



Changing the list is safer than copying and editing seven separate commands.

1.6.2 Repeating Two Commands

Braces group several commands into the body of one loop.

```
\begin{tikzpicture}
  \foreach \x in {0,1,2,3,4}
  {
    \draw (\x,0) -- (\x,0.3);
    \draw (\x,-0.1) -- (\x,-0.2);
  }
\end{tikzpicture}
```

See



Each command inside the braces needs its own semicolon.

1.7 Debugging Workshop

Begin with syntax. A coordinate change cannot repair a missing semicolon or an incorrect environment name.

1.7.1 Missing Semicolon

Incorrect:

```
\draw (0,0) -- (3,1)
```

Correct:

```
\draw (0,0) -- (3,1);
```

1.7.2 Drawing Outside the Environment

Incorrect:

```
\draw (0,0) -- (3,1);
```

Correct:

```
\begin{tikzpicture}  
  \draw (0,0) -- (3,1);  
\end{tikzpicture}
```

1.7.3 Mismatched Coordinate Punctuation

Incorrect:

```
\draw (0,0] -- (3,1);
```

Correct:

```
\draw (0,0) -- (3,1);
```

Coordinates use parentheses. Options use square brackets.

1.7.4 A Loop Variable Without Its Backslash

Incorrect:

```
\foreach \x in {0,1,2,3}  
  \draw (x,0) -- (x,0.3);
```

Correct:

```
\foreach \x in {0,1,2,3}  
  \draw (\x,0) -- (\x,0.3);
```

Use this checking order: environment, command name, option position, coordinate punctuation, path operators, loop variables, semicolon. Only then inspect the numerical coordinates.

1.8 Worked Project: A Coordinate Practice Board

The project combines axes, repeated marks, a broken path, and arrows without using any commands from later chapters.

1.8.1 Stage 1: Draw the Frame

```
\begin{tikzpicture}
  \draw[>-] (-0.5,0) -- (5.5,0);
  \draw[>-] (0,-0.5) -- (0,3.5);
\end{tikzpicture}
```

1.8.2 Stage 2: Add Repeated Marks

```
\foreach \x in {1,2,3,4,5}
  \draw (\x,0.1) -- (\x,-0.1);

\foreach \y in {1,2,3}
  \draw (0.1,\y) -- (-0.1,\y);
```

1.8.3 Stage 3: Add Two Paths

```
\draw[->]
  (0,0) -- (1,0.8) -- (2,0.5) -- (4,2.5);

\draw[->]
  (0,0) -- (1.5,2.5) -- (3,1.5) -- (5,3);
```

1.8.4 The Complete Picture

```
\begin{tikzpicture}
  \draw[->] (-0.5,0) -- (5.5,0);
  \draw[->] (0,-0.5) -- (0,3.5);

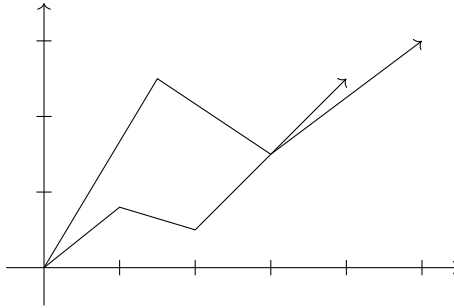
  \foreach \x in {1,2,3,4,5}
    \draw (\x,0.1) -- (\x,-0.1);

  \foreach \y in {1,2,3}
    \draw (0.1,\y) -- (-0.1,\y);

  \draw[->]
    (0,0) -- (1,0.8) -- (2,0.5) -- (4,2.5);

  \draw[->]
    (0,0) -- (1.5,2.5) -- (3,1.5) -- (5,3);
\end{tikzpicture}
```

See



Design

The source is organized by purpose: frame, marks, first path, second path. This kind of organization matters even in a small picture and becomes essential in larger diagrams.

1.9 Exercises

Try each exercise before reading the solutions.

1. Draw a segment from $(-2, 1)$ to $(3, 1)$.
2. Draw an arrow from $(2, 2)$ to $(-1, -1)$.
3. Draw one continuous path through five coordinates.
4. Reverse the direction of an existing arrow without changing its length.
5. Use a loop to draw eight vertical marks spaced one-half unit apart.
6. Use a loop to draw five horizontal marks along a vertical line.
7. Draw two coordinate axes and place four marks on each axis.
8. Modify the worked project by moving exactly one point in each path.

1.10 Solutions

The following are sample solutions. Many other coordinate choices are equally correct.

1.10.1 Exercises 1–4

```
\begin{tikzpicture}
  \draw (-2,1) -- (3,1);
\end{tikzpicture}
```

```
\begin{tikzpicture}
  \draw[->] (2,2) -- (-1,-1);
\end{tikzpicture}
```

```
\begin{tikzpicture}
  \draw
    (0,0) -- (1,1) -- (2,0.5) -- (3,1.5) -- (4,0);
\end{tikzpicture}
```

```
\begin{tikzpicture}
  \draw[->] (3,0) -- (0,0);
\end{tikzpicture}
```

1.10.2 Exercises 5–7

```
\begin{tikzpicture}
  \foreach \x in {0,0.5,1,1.5,2,2.5,3,3.5}
    \draw (\x,0) -- (\x,0.3);
\end{tikzpicture}
```

```
\begin{tikzpicture}
  \foreach \y in {0,1,2,3,4}
    \draw (0,\y) -- (0.3,\y);
\end{tikzpicture}
```

```
\begin{tikzpicture}
  \draw[>-] (-0.5,0) -- (4.5,0);
  \draw[>-] (0,-0.5) -- (0,4.5);
  \foreach \x in {1,2,3,4}
    \draw (\x,0.1) -- (\x,-0.1);
  \foreach \y in {1,2,3,4}
    \draw (0.1,\y) -- (-0.1,\y);
\end{tikzpicture}
```

For Exercise 8 there is no single solution. Move one coordinate, compile, and describe exactly how the corresponding segment changed.

1.11 ChatGPT Workshop

Ask ChatGPT to respect the chapter boundary. A narrowly defined prompt usually produces a more useful answer than a general request for a diagram.

Using ChatGPT

1. Explain `\draw[->] (-1,0) -- (2,1);` one part at a time.
2. Give me five coordinate exercises using only `\draw`, `--`, and `->`.
3. I will paste a Chapter 1 TikZ picture. Predict its geometry before I compile it.
4. Check this picture for environment, punctuation, option-placement, and semicolon errors. Do not redesign it.
5. Use one `\foreach` loop in place of my repeated drawing commands. Use no commands from later chapters.
6. Change one coordinate at a time and explain the effect of each change.

1.12 Final Checklist

Before continuing to Chapter 2, check that you can

1. place drawing commands inside `tikzpicture`;
2. read a coordinate as horizontal and vertical information;
3. join two or more coordinates with `--`;
4. place `->` in the option list and control arrow direction;
5. end every drawing command with a semicolon;
6. use `\foreach` to repeat one or more commands; and
7. debug syntax before changing geometry.